2.3. Технология создания таблиц и ограничений. Создание диаграммы базы данных (Практическая работа №3)

Создание таблиц и ограничений

Таблицы представляют собой объекты базы данных, используемые непосредственно для хранения всех данных. Одним из самых главных правил организации баз данных является то, что в одной таблице должны храниться данные лишь об одном конкретном типе сущности (например, клиенты, товары, заказы и т. п.).

Данные в таблицах организованы по полям и записям. Поля (или столбцы таблицы) содержат определенный тип информации, например, фамилию, адрес, телефонный номер. Запись (или строка таблицы) - группа связанных полей, содержащих информацию об отдельном экземпляре сущности.

Любое поле таблицы характеризуется как минимум тремя обязательными свойствами:

–Имя столбца. Реализует способ обращения к конкретному полю в таблице. Рекомендуется всегда присваивать полям смысловые имена.

–Тип данных. Определяет, информация какого типа может храниться в данном поле.

–Разрешить значения null. Определяет, допустимо ли для данного поля отсутствие фактических данных, для обозначения которого используется так называемый маркер пустого значения null.

Типы данных

При выборе типа данных для столбца следует отдавать предпочтение типу, который позволит хранить любые возможные для этого столбца значения и занимать при этом минимальное место на диске. Типы данных в MS SQL Server можно разделить на восемь категорий:

1. Целочисленные данные

– bit (1 байт). Может хранить только значения 0, 1 или null (пустое значение, сообщающее об отсутствии данных). Его удобно использовать в качестве индикатора состояния – включено/выключено, да/нет, истина/ложь.

– tinyint (1 байт).Целые значения от 0 до 255.

– smallint (2 байта). Диапазон значений от -2^{15} (-32768) до 2^{15} (3767).

– int (4 байта). Может содержать целочисленные данные от -2^{31} (-2147483648) до 2^{31} (21474833647).

– bigint (8 байт). Включает в себя данные от -2^{63} (9223372036854775808) до 2^{63} (9223372036854775807). Удобен для хранения очень больших чисел, не помещающихся в типе данных int.

2. Текстовые данные

– char. Содержит символьные не Unicode-данные фиксированной длины до 8000 знаков.

– varchar. Содержит символьные не Unicode-данные переменной длины до 8000 знаков.

– nchar. Содержит данные Unicode фиксированной длины до 4000 символов. Подобно всем типам данных Unicode его удобно использовать для хранения небольших фрагментов текста, которые будут считываться разноязычными клиентами.

– nvarchar. Содержит данные Unicode переменной длины до 4000 символов *(рекомендуется использовать этот тип из перечня текстовых*).

3. Десятичные данные

– decimal. Содержит числа с фиксированной точностью от -1038-1 до 1038-1. Он использует два параметра: точность и степень. Точностью называется общее количество знаков, хранящееся в поле, а степень – это количество знаков справа от десятичной запятой.

– numeric. Это синоним типа данных decimal – они идентичны.

4. Денежные типы данных

– money (8 байт). Содержит денежные значения от -263 до 263 с десятичной точностью от денежной единицы. Удобен для хранения денежных сумм, превышающих 214768,3647.

– smallmoney (4 байта). Содержит значения от -214748,3648 до 214748,3647 с десятичной точностью.

5. Данные с плавающей точкой

– float. Содержит числа с плавающей запятой от -1,79E+38 до 1,79E+38.

– real. Содержит числа с плавающей запятой от -3,40E+38 до 3,40E+38.

6. Типы данных даты и времени

– datetime (8 байт). Содержит дату и время в диапазоне от 1 января 1753 года до 31 декабря 9999 года с точностью 3,33 мс.

– smalldatetime (4 байта). Содержит дату и время, начиная от 1 января 1900 года и заканчивая 6 июнем 2079, с точностью до 1 минуты.

7. Двоичные типы данных

– binary. Содержит двоичные данные фиксированной длины до 8000 байт.

– varbinary. Содержит двоичные данные переменной длины до 8000 байт.

8. Специализированные типы данных

– sql_variant. Используется для хранения значения с различными типами данных.

– timestamp. Используется для установки временных меток записей при вставке, которые соответствующим образом обновляются. Удобен для отслеживания изменений в данных.

– uniqueidentifier. Глобальный уникальный идентификатор.

– xml. Используется для хранения целых документов или фрагментов XML.

Создание пользовательских типов данных

SQL Server позволяет на основе системных типов данных создавать пользовательские типы со всеми предварительно заданными параметрами, включая все ограничения и умолчания. В качестве примера создадим тип данных phone, который будет использоваться в таблице Customer для хранения телефонного номера клиента. Для его создания воспользуемся графическим интерфейсом утилиты Management Studio.

1. В дереве обозревателя объектов раскройте папки «Базы данных – Sales – Программирование - Типы». В контекстном меню узла «Определяемые пользователем типы данных» выберите команду «Создать определяемый пользователем тип данных».

2. В появившемся окне в текстовом поле «Имя» введите phone. В раскрывающемся списке «Тип данных» выберите nchar. В качестве длины введите 10. Отметьте параметр «Разрешить значения null», чтобы иметь возможность не указывать телефонный номер при добавлении нового клиента.

3. В секции «Привязки» оставьте пустые значения и щелкните на кнопке Ok. Созданный пользовательский тип данных должен появиться в дереве обозревателя объектов.

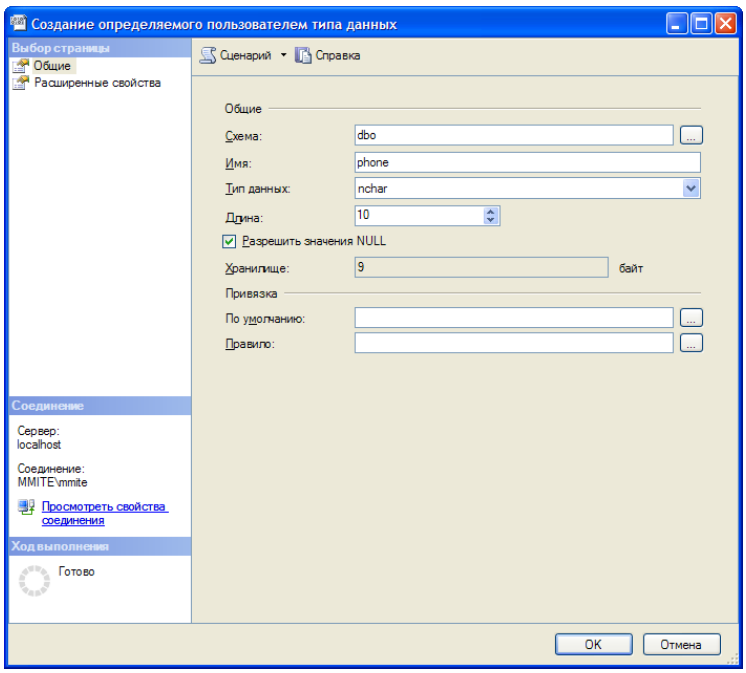


Рисунок 2.3.1

Создание таблиц

Создадим в базе данных Sales пять таблицы. Первая таблица, Customer, будет хранить информацию о клиентах, вторая таблица City – справочник городов, третья, Product, - информацию о товарах, четвертая, Order, будет содержать подробную информацию о заказах и пятая, OrdItem, - о составе заказа (перечне товаров входящих в заказ). Ниже представлены все поля этих таблиц и их основные свойства.

Таблица 2.3.1

Имя столбца	Тип данных	Разрешить null	Описание
Customer			
IdCust	int, identity	нет	Уникальный идентификационный номер клиента, на который можно ссылаться в других таблицах
FName	nvarchar(20)	нет	Имя клиента
LName	nvarchar(20)	нет	Фамилия клиента
IdCity	int	нет	Ссылка на номер города
Address	nvarchar(50)	нет	Адрес клиента

Продолжение таблицы 2.3.1

Имя столбца	Тип данных	Разрешить null	Описание
Zip	nchar(5)	нет	Почтовый индекс клиента
Phone	phone	да	Телефонный номер клиента
City			
IdCity	int, identity	нет	Уникальный идентификационный номер города
CityName	nvarchar(20)	нет	Название города
Product			
IdProd	int, identity	нет	Уникальный идентификационный номер для каждого товара
Description	nvarchar(100)	нет	Короткое текстовое описание товара
InStock	int	нет	Количество единиц продукта на складе
Order			
IdOrd	int, identity	нет	Уникальный идентификационный номер заказа
IdCust	int	нет	Ссылка на номер клиента
OrdDate	smalldatetime	нет	Дата и время размещения заказа
OrdItem			
IdOrd	int	нет	Ссылка на номер заказа
IdProd	int	нет	Ссылка на номер товара
Qty	int	нет	Количество единиц товара в заказе
Price	money	нет	Цена товара

Таблицы можно создавать как в графическом интерфейсе (в утилите Management Studio), так и с помощью кода T-SQL. Воспользуемся самым простым, графическим способом. Сначала создадим таблицу Customer:

1. В дереве обозревателя объектов в базе данных Sales в контекстном меню узла «Таблицы» выберите команду «Создать таблицу…». В рабочей области должна появиться вкладка с конструктором таблиц.

2. В первую строку в столбце «Имя столбца» введите IdCust, в столбце «Тип данных» выберите int. Убедитесь что параметр «Разрешить значения null» отключен.

3. В нижней половине экрана в разделе «Свойства столбцов» введите описание поля и измените значение параметра «Спецификация идентификатора / (Идентификатор)» на «Да» для того чтобы значения номера клиента формировались автоматически. Свойство «Идентифицирующий столбец» (Identity), обычно используемое совместно с типом данных int, предназначено для автоматического приращения значения на единицу при добавлении каждой новой записи. К примеру, клиент, добавленный в таблицу первым, будет иметь значение идентификатора 1, вторым – 2, третьим – 3, и т.д.

4. Аналогичным образом введите описания всех остальных полей и закройте окно конструктора таблиц. Введите в качестве имени таблицы Customer. Вновь созданная таблица должна появиться в дереве обозревателя объектов в папке «Таблицы».

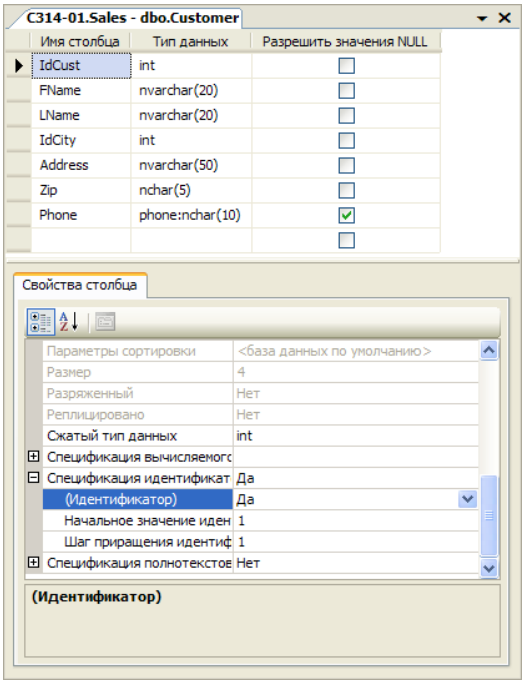


Рисунок 2.3.2

Задание для самостоятельной работы: В соответствие с вышеприведенным описанием создайте оставшиеся четыре таблицы: City, Product, Order и OrdItem. Внесите в отчет итоги этой работы.

Создание ограничений

Перед тем как начать работать с таблицами следует ограничить вводимые в них данные в целях обеспечения так называемой целостности данных, т. е. ограничить возникновение в базе данных некорректных или противоречивых данных вследствие добавления, изменения или удаления какой-либо записи, например, ввод отрицательной цены или количества товара. Существует четыре типа целостности данных: доменная, сущностная, ссылочная и пользовательская (или бизнес-правила). Рассмотрим основные инструменты, предоставляемые в SQL Server для их реализации.

Обеспечение доменной целостности. Ограничение диапазона данных, вводимых пользователем в поле. Основными инструментами обеспечения доменной целостности являются ограничения проверки и значения по умолчанию.

Использование проверочных ограничений

Ограничения на проверку используются для ограничения данных, принимаемых полем, даже если они имеют корректный тип. Например, поле Zip (почтовый индекс) имеет тип nchar(5), т.е. чисто теоретически оно может принимать буквы. Это может стать проблемой, поскольку не существует почтовых индексов с буквами. Рассмотрим, как создать ограничение на проверку, запрещающее вводить в это поле буквы.

1. В контекстном меню папки «Ограничения» таблицы Customer выберите команду «Создать ограничение».

2. В открывшемся окне «Проверочные ограничения» заполните следующие поля:

– Имя: CK_Zip
– Выражение: ([zip] like '[0-9][0-9][0-9][0-9][0-9]'). Данное выражение описывает ограничение, принимающее пять символов, которыми могут быть только цифры от 0 до 9.
– Описание: Ограничение на значения почтового индекса

3. Щелкните на кнопке «Закрыть» и закройте конструктор таблиц (он был открыт, когда вы начали создавать ограничение) с сохранением изменений.

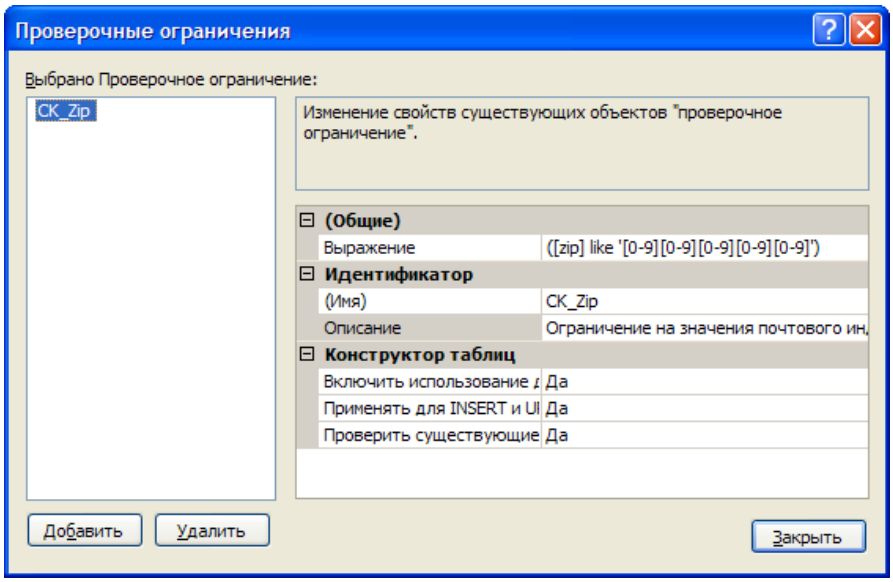


Рисунок 2.3.3

Задание для самостоятельной работы: Создайте ограничения для полей InStock таблицы Product и Qty, Price таблицы OrdItem, запрещающие ввод в них отрицательных значений. В данном случае выражение проверки будет иметь вид (Имя поля > 0) для полей Qty, Price и (Instock>=0) для столбца InStock.

Использование значений по умолчанию

Установка для полей значений по умолчанию это отличный способ избавить пользователя от излишней работы, если значения этих полей во всех записях, как правило, принимают одни и те же значения. Так в таблице заказов Order вполне логично определить по умолчанию значение поля OrdDate (дата заказа) в виде текущей даты. В этом случае при добавлении записи о новом заказе в случае пропуска этого поля оно будет автоматически заполняться значением системной даты. Для создания такого свойства выполните следующие шаги:

1. Раскройте папку «Столбцы» таблицы Order и в контекстном меню поля «OrdDate» выберите команду «Изменить».

2. В свойстве столбца «Значение или привязка по умолчанию» введите getdate(). Эта функция T-SQL возвращает текущую системную дату.

3. Щелкните на кнопке Сохранить и выйдите из конструктора таблиц.

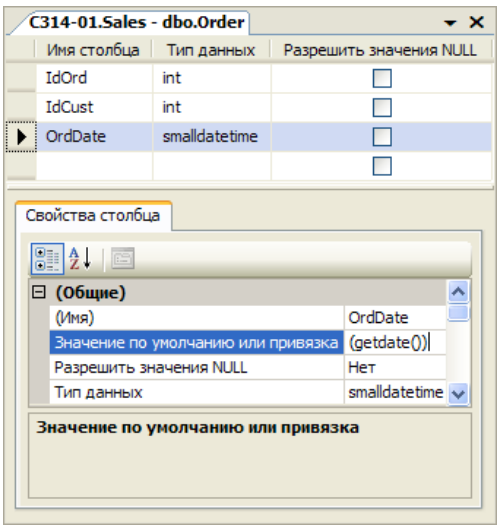


Рисунок 2.3.4

Задание для самостоятельной работы: Установите для поля InStock (количество единиц продукта на складе) таблицы Product в качестве значения по умолчанию ноль.

Обеспечение сущностной целостности. Обеспечение гарантии уникальности записей в таблицах и предотвращение их дублирования. Основными инструментами обеспечения целостности сущностей являются первичные ключи и ограничения уникальности.

Создание первичных ключей

Первичный ключ используется для обеспечения гарантии уникальности каждой записи в таблице. Он состоит из одного (простой ключ) или нескольких (составной ключ) столбцов с гарантированно уникальными значениями для каждой записи таблицы. Если пользователь попытается ввести в поля первичного ключа дублирующее значение будет сгенерирована ошибка и модификация данных будет отменена.

В качестве примера создадим первичный ключ для таблицы Customer. В данном случае идеальным кандидатом на роль первичного ключа выступает столбец IdCust, поскольку значения, содержащиеся в нем, являются уникальными по определению (для него установлено свойство identity). Следует отметить, что в качестве первичного ключа могут быть взяты и реальные атрибуты клиента, например, ИНН, номер страхового свидетельства, серия и номер паспорта вместе взятые (пример составного ключа), но использование различных разновидностей, так называемых, суррогатных ключей (identity, uniqueidentifier) обеспечивает большую степень сущностной целостности (поскольку реальные атрибуты могут все же со временем измениться) и является распространенной практикой. Для создания первичного ключа в таблице Customer выполните следующие шаги:

1. В контекстном меню таблицы Customer выберите команду «Проект».

2. В окне конструктора таблиц щелкните правой кнопкой мыши на поле IdCust и выберите команду «Задать первичный ключ» или нажмите кнопку  на панели инструментов. Обратите внимание на то, что слева от поля IdCust теперь отображается значок ключа, указывающий, что поле является первичным ключом.

3. Закройте конструктор таблиц с сохранением изменений

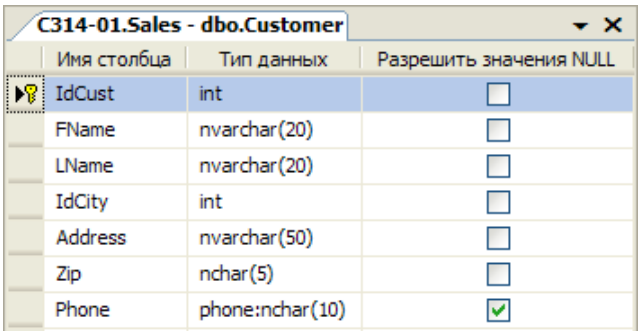


Рисунок 2.3.5

Задание для самостоятельной работы: Аналогичным образом создайте первичные ключи для остальных таблиц в соответствие с ниже приведенной таблицей 2.3.2.

Таблица 2.3.2

Таблица	Первичный ключ
City	IdCity
Product	IdProd
Order	IdOrd
OrdItem	IdOrd, IdProd (для выбора нескольких столбцов при установке составного ключа воспользуйтесь клавишами Shift или Ctrl)

Использование ограничений на уникальность

Между ограничениями первичного ключа и ограничениями на уникальность существует два отличия. Первое состоит в том, что первичные ключи используются вместе с внешними ключами для обеспечения целостности ссылок (рассматривается в следующем разделе). Второе отличие заключается в том, что ограничения на уникальность позволяют вставлять в его поля пустые значения (null), чего нельзя делать с первичными ключами. Во всем остальном они служат одной цели – обеспечить уникальность данных, вставляемых в поле. Ограничение на уникальность следует использовать в тех случаях, когда нужно гарантировать, что дублирующие значения не будут добавляться в поле, не являющееся частью первичного ключа, в частности, все потенциальные ключи должны быть организованы в виде ограничений уникальности. Хорошим примером такого поля, требующего ограничение на уникальность, является поле ИНН или серия и номер паспорта, поскольку эти поля должны быть уникальными у каждого человека. Такого идеального кандидата на роль уникального ограничения в нашей таблице Customer нет. Поэтому создадим его по полю Phone, которое также повторяться у разных клиентов не должно.

1. Для открытия конструктора таблиц в контекстном меню таблицы Customer выберите команду «Проект». На панели инструментов нажмите на кнопку «Управление индексами и ключами»  .

2. В открывшемся окне «Индексы и ключи» щелкните кнопку «Добавить» и введите следующие параметры для нового уникального ключа:

– Столбцы: Phone
– Тип: Уникальный ключ
– (Имя): CK_Phone

3. Закройте конструктор таблиц с сохранением изменений.

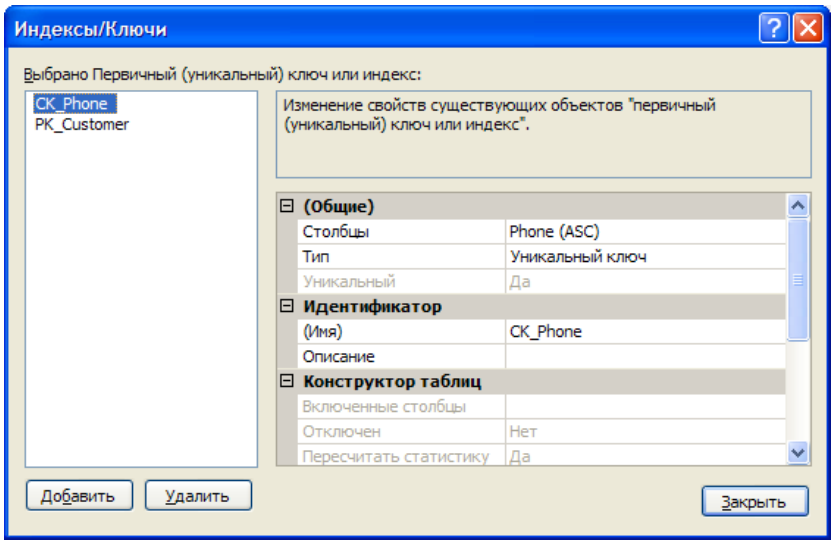


Рисунок 2.3.6

Задание для самостоятельной работы: Аналогичным образом создайте ограничение уникальности по полю CityName таблицы City, чтобы обеспечить отсутствие в справочнике городов с одинаковыми названиями, а также по полю Description таблицы Product, чтобы иметь возможность отличить один товар от другого.

Обеспечение целостности ссылок

Сейчас в базе данных Sales имеются пять таблиц, которые тесно взаимосвязаны между собой и соответственно данные содержащиеся в них должны быть согласованы и непротиворечивы. Например, в таблице Order не должно быть записей о заказах для клиента, данные о котором отсутствуют в таблице Customer. Чтобы гарантировать отсутствия в базе данных таких записей необходимо обеспечить целостность ссылок.

Суть обеспечения целостности ссылок очевидна из названия: данные в одной таблице, ссылающиеся на данные из другой таблицы, защищены от некорректного обновления. В терминологии SQL Server это называется *декларативной ссылочной целостностью* и достигается путем связывания первичного ключа одной из таблиц с внешним ключом другой таблицы (создается так называемое ограничение внешнего ключа).

Внешний ключ используется в комбинации с первичным для связывания двух таблиц по общему столбцу (столбцам). К примеру, можно связать таблицы Customer и Order по столбцу IdCust, который присутствует в обеих таблицах. Поскольку поле IdCust таблицы Customer является его первичным ключом можно использовать поле IdCust таблицы Order в качестве внешнего ключа, который свяжет эти две таблицы. После организации такого ограничения будет невозможно добавить запись в таблицу Order, если в таблице Customer нет записи с соответствующим значением IdCust. Кроме того, при отсутствии каскадирования (рассматривается в следующем разделе) невозможно удалить запись из таблицы Customer при наличии связанных с ней записей в таблице Order, поскольку нельзя оставлять заказ без информации о клиенте. Для создания описанного ограничения внешнего ключа в Management Studio выполните следующие шаги:

1. В контекстном меню папки «Ключи» таблицы Order выберите команду «Создать внешний ключ…».

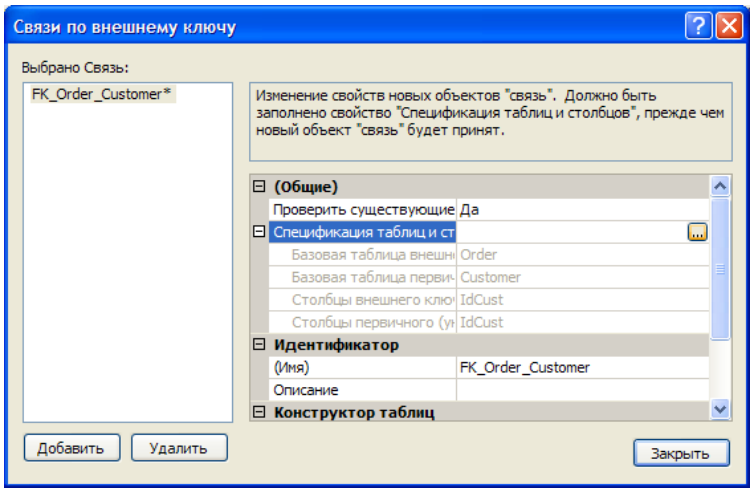


Рисунок 2.3.7

2. В открывшемся окне «Отношения внешнего ключа» заполните следующие поля:

– (Имя): FK_Order_Customer

– Спецификация таблиц и столбцов: Для заполнения данного блока щелкните на кнопке с многоточием и в появившемся окне «Таблицы и столбцы» в качестве таблицы первичного ключа выберите Customer, а полей связи - IdCust.

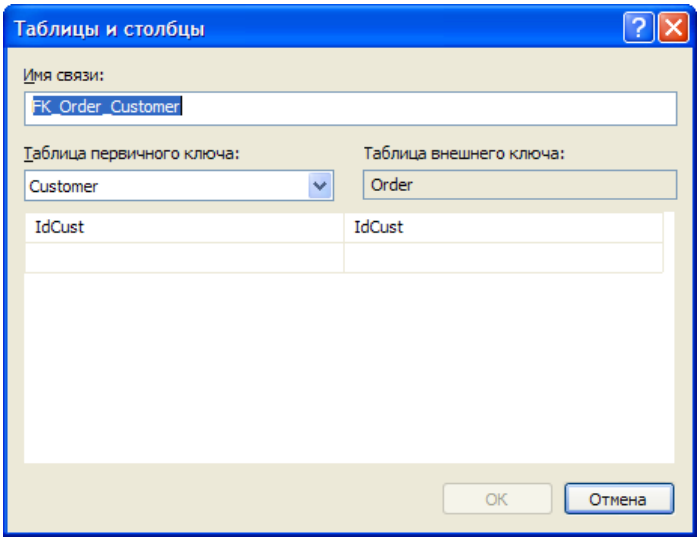


Рисунок 2.3.8

3. Закройте все открывшиеся окна с сохранением изменений.

Использование каскадной ссылочной целостности

При наличии ограничения внешнего ключа с параметрами по умолчанию вы не можете удалить запись или изменить значение первичного ключа главной таблицы в случае наличия связанных записей в подчиненной таблице (в которой организовано ограничение внешнего ключа). Однако это поведение можно изменить, используя *каскадную ссылочную целостность*.

Настроить правила каскадирования можно при создании ограничения внешнего ключа в окне «Связи по внешнему ключу» изменяя значения параметров «Правило

обновления» и «Правило удаления» блока «Спецификация INSERT и UPDATE». Оба этих параметра могут содержать четыре значения, описанные в следующей таблице 2.3.3.

Таблица 2.3.3

Настройка	Правило удаления	Правило обновления
Нет действия	Невозможно удалить в главной таблице строку, на которую есть ссылки в подчиненной	Невозможно обновить значения полей первичного ключа главной таблицы при наличии связанных записей в подчиненной
Каскадно	При удалении строки в главной таблице все связанные строки в подчиненной также будут удалены	При обновлении значений полей первичного ключа главной таблицы соответствующим образом будут изменены и их значения во всех связанных строках подчиненной таблицы
Присвоить Null	При удалении строки в главной таблице во всех связанных строках подчиненной полям вторичного ключа будет присвоено значение Null	При обновлении значений полей первичного ключа главной таблицы во всех связанных строках подчиненной таблицы полям вторичного ключа будет присвоено значение Null
Присвоить значение по умолчанию	При удалении строки в главной таблице во всех связанных строках подчиненной полям вторичного ключа будут присвоены значения по умолчанию	При обновлении значений полей первичного ключа главной таблицы во всех связанных строках подчиненной таблицы полям вторичного ключа будут присвоены значения по умолчанию

Задание для самостоятельной работы: Создайте ограничение внешнего ключа FK_OrdItem_Order в таблице OrderItem для связи таблиц Order и OrderItem по полю IdOrd. При этом настройте правило каскадного удаления, установив в качестве параметра «Спецификация INSERT и UPDATE\Правило удаления» значение «Каскадно», что приведет к автоматическому удалению всех товаров из заказа при удалении самого заказа.

Создание диаграмм баз данных

Диаграммы базы данных представляют собой графическое отображение схемы (целиком или частично) базы данных с таблицами и столбцами, а также связей между ними. Создадим диаграмму базы данных:

1. В контекстном меню папки «Диаграммы базы данных» выберите команду «Создать диаграмму базы данных».

2. В диалоговом окне «Добавление таблиц» выберите все таблицы и нажмите на кнопку «Добавить».

3. Добавив таблицы, щелкните на кнопке «Закрыть» и вы увидите созданную диаграмму базы данных (на рисунке представлен окончательный вид диаграммы: некоторые связи у вас могут отсутствовать).

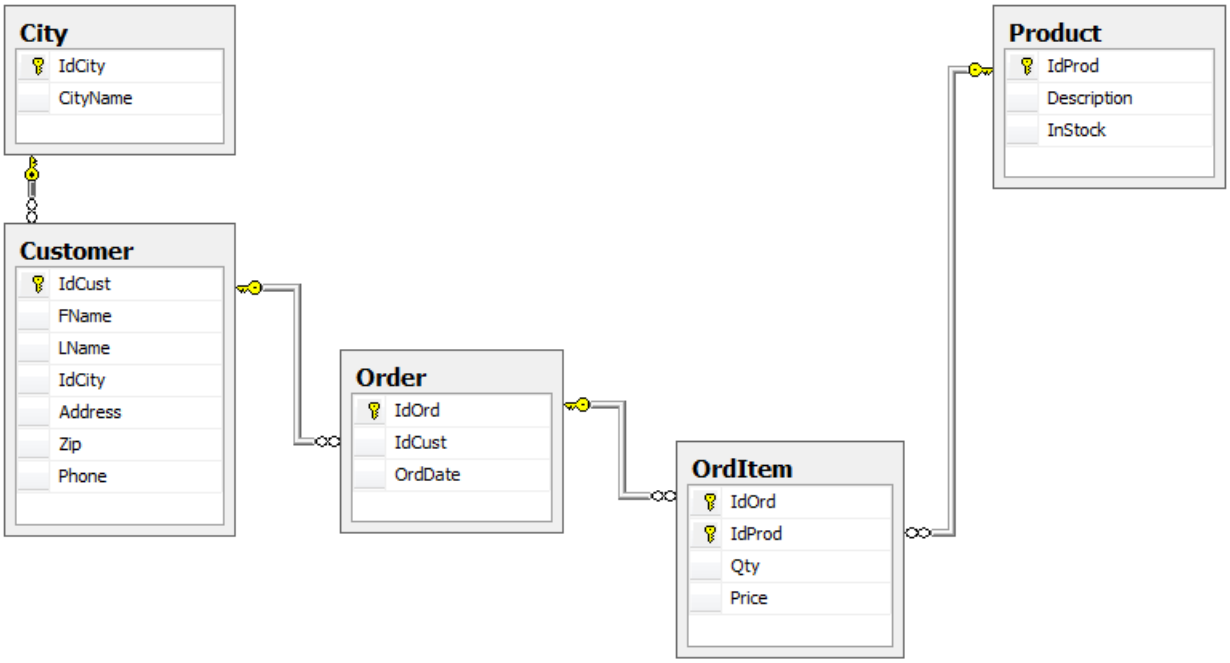


Рисунок 2.3.9

Используя диаграмму базы данных ограничения внешнего ключа можно создавать значительно быстрее: лишь перетаскивая поля из одной таблицы в другую. В качестве примера создадим внешний ключ в таблице Customer по полю IdCity для связи с таблицей City:

1. Выделите в таблице City поле IdCity и, не отпуская кнопку мыши, перетащите его на поле IdCity таблицы Customer.

2. В диалоговых окнах «Таблицы и столбцы» и «Связь по внешнему ключу» примите настройки по умолчанию.

3. Сохраните диаграмму базы данных под именем ILM.

Расположите таблицы в канонической форме (главные таблицы выше подчиненных) в соответствии с вышеприведенным рисунком.

Задание для самостоятельной работы: Аналогичным образом создайте связь между таблицами Product и OrdItem по полю IdProduct. Окончательный список связей между таблицами со всеми их характеристиками представлен в следующей таблице 2.3.4:

Таблица 2.3.4

Главная таблица	Подчиненная таблица	Поле связи (внешний ключ)	Правила каскадирования
City	Customer	IdCity	Без действия
Customer	Order	IdCust	Без действия
Order	OrdItem	IdOrd	Каскадное удаление
Product	OrdItem	IdProd	Без действия

Задание для самостоятельной работы: После настройки всех ограничений можно наполнить таблицы данными. Для этого в контекстном меню таблицы выберите команду «Изменить первые 200 строк» и появившейся в рабочей области вкладке введите новые записи, заполняя все необходимые столбцы. В процессе внесения данных проверьте работоспособность всех созданных ранее ограничений:

– Ограничений проверки: попробуйте ввести в поле Zip (почтовый индекс) таблицы Customer нечисловые значения, а в поля InStock таблицы Product и Qty, Price таблицы OrdItem - отрицательные.

– Значений по умолчанию: убедитесь, что при пропуске полей OrdDate и InStock таблиц Order и Product для них устанавливаются значения по умолчанию в виде текущей системной даты и нуля соответственно.

– Ограничений первичного и уникального ключа: попробуйте ввести в таблицы записи с дублирующими значениями первичного или уникального ключа.

– Ограничений внешнего ключа: попробуйте ввести несогласованные данные в связанные таблицы, например, заказ для несуществующего клиента или удалить запись из любой главной таблицы при наличии связанных записей в подчиненной при отсутствии правил каскадирования.

– Правил каскадирования: убедитесь, что при удалении записи из таблицы Order все связанные записи из таблицы OrdItem удаляются автоматически.